Planeamiento de movimiento en robótica multi-agente

Presentado por:

Leonardo López Ortiz

Presentado a:

German Holguín PhD

Métodos y modelos computacionales

Maestría en Ingeniería Eléctrica

Universidad Tecnológica de Pereira

Pereira

2024

Introducción

El presente informe describe la estrategia y metodología utilizada para resolver el problema de encontrar caminos óptimos en un grafo mediante el uso del algoritmo de Dijkstra. Este método se seleccionó tras evaluar diversos algoritmos en un proyecto previo, donde Dijkstra demostró ser el más eficaz al encontrar caminos más cortos, especialmente en grafos ponderados con pesos no negativos. La solución presentada aborda también la simulación multi-agente, donde varios robots "tortugas" navegan simultáneamente a través del mapa generado con obstáculos.

Objetivos:

El objetivo es encontrar las rutas más cortas entre puntos de inicio y destino en un grafo, mientras se consideran las siguientes condiciones:

- Los grafos pueden contener "obstáculos" que restringen la navegación.
- El sistema debe soportar múltiples agentes ("tortugas") navegando simultáneamente sin interferencias.
- Se requiere evaluar el comportamiento tanto en grafos con soluciones viables (conexión entre puntos) como en aquellos donde la solución no existe (grafo desconectado).

Metodología de Solución

1. Estructura del Problema: El problema puede formularse como un problema de optimización en un grafo representado por:

V: Conjunto de vértices del grafo.

E: Conjunto de aristas que conectan pares de vértices, con pesos asociados (representan distancias o costos).

s: Nodo de inicio.

t: Nodo de destino.

El objetivo es encontrar la secuencia de vértices que minimice el costo total desde **s** hasta **t**.

2. Algoritmo de solución: Dijkstra

El algoritmo de Dijkstra se seleccionó debido a su capacidad para garantizar soluciones óptimas en grafos ponderados con pesos no negativos. Su metodología consiste en:

- Inicializar todos los vértices con una distancia infinita, excepto el nodo de inicio, que tiene una distancia de 0.
- Usar una cola de prioridad para explorar iterativamente el nodo más cercano (con menor costo acumulado).
- Actualizar las distancias de los vecinos del nodo explorado si se encuentra un camino más corto.
- Repetir hasta alcanzar el nodo de destino o explorar todos los nodos conectados.

3. Extensión a escenarios multi-agente

Para soportar múltiples tortugas navegando simultáneamente:

- Se generan caminos independientes para cada tortuga mediante ejecuciones separadas del algoritmo de Dijkstra.
- Cada tortuga tiene un punto de inicio y destino únicos seleccionados aleatoriamente.
- Se asignan colores únicos a cada tortuga para diferenciar sus rutas en la visualización.

4. Evaluación de Casos

Se contemplaron dos escenarios principales:

- Con solución: Grafos donde existe al menos un camino viable entre los nodos de inicio y destino. Se evaluaron configuraciones con diferentes densidades de obstáculos.

- Sin solución: Grafos donde los nodos de inicio y destino están desconectados debido a obstáculos.

5. Implementación

El algoritmo se implementó en Python, con las siguientes características:

- Visualización Dinámica: Los movimientos de las tortugas se representan paso a paso.
- Simulación Paralela: Uso de hilos para ejecutar simultáneamente la planificación y la visualización.
- Configurabilidad: Soporte para ajustar el tamaño del grafo y la densidad de obstáculos.

Resultados

Caso 1: Grafo Conexo

Tamaño del grafo: 10x10

Obstáculos: de 10% hasta 30%

Los nodos de colores representan donde comienza cada tortuga individual y todos los nodos rojos, el objetivo de llegada.

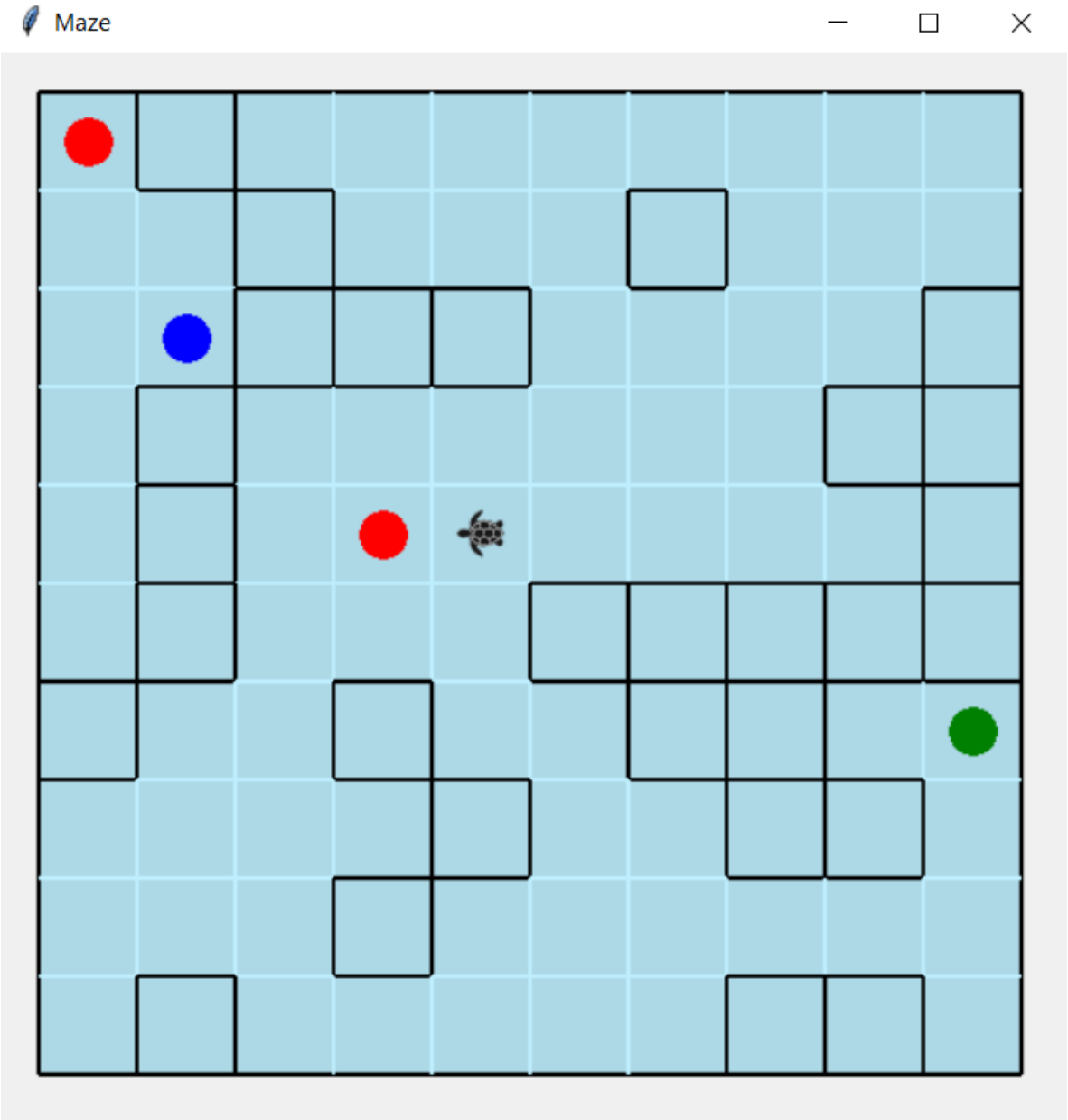

Figura 1. Prueba 1 con generación aleatoria de nodos, tortugas y obstáculos.

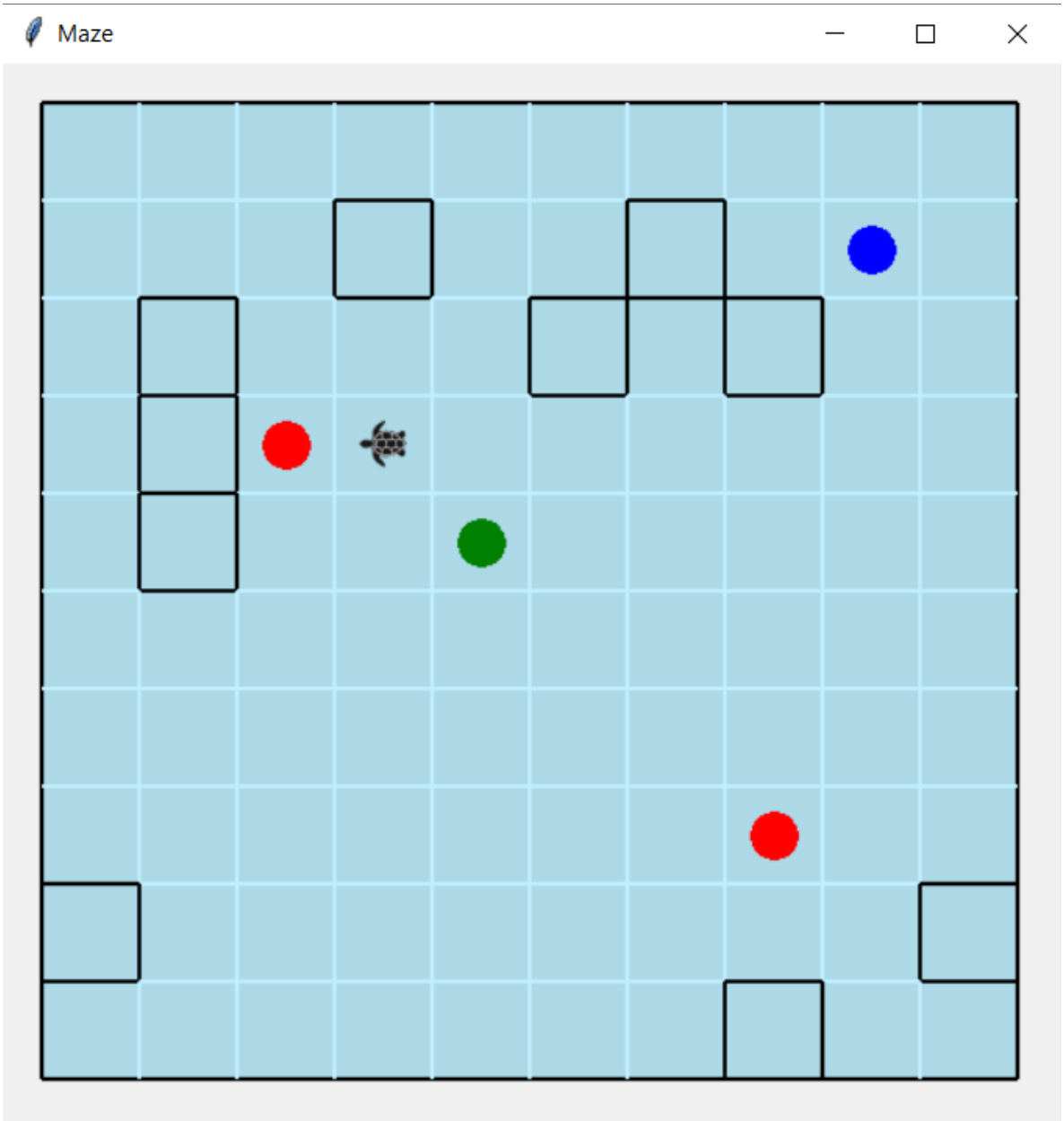

Figura 2. Prueba 2 con generación aleatoria de nodos, tortugas y obstáculos.

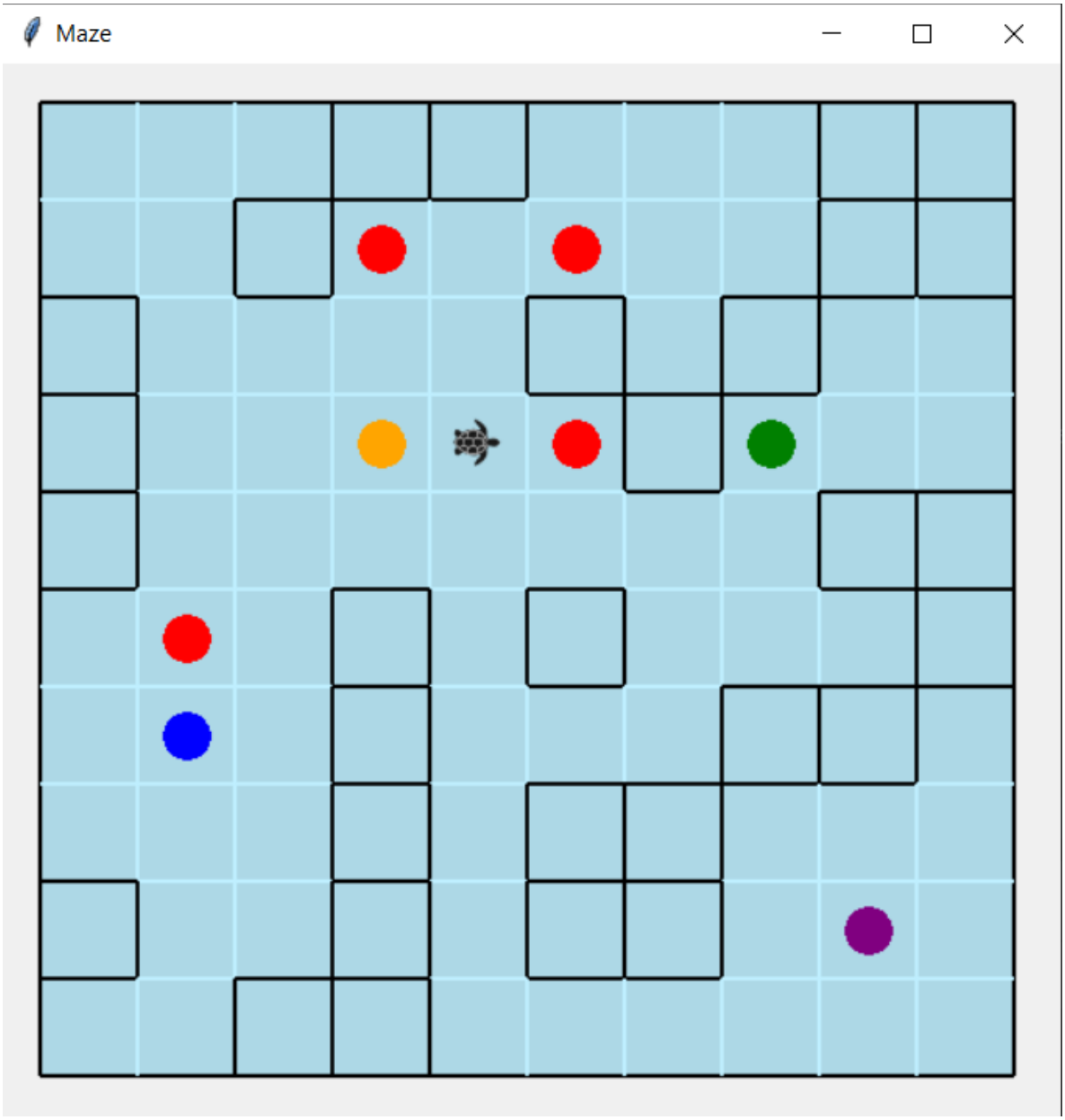

Figura 3. Prueba 3 con generación aleatoria de nodos, tortugas y obstáculos.

Resultado: Todas las tortugas completaron las rutas con éxito.

Caso 2: Grafo Desconexo

Tamaño del grafo: 15x15

Obstáculos: 10% hasta 70%

Los nodos de colores representan donde comienza cada tortuga individual y todos los nodos rojos, el objetivo de llegada

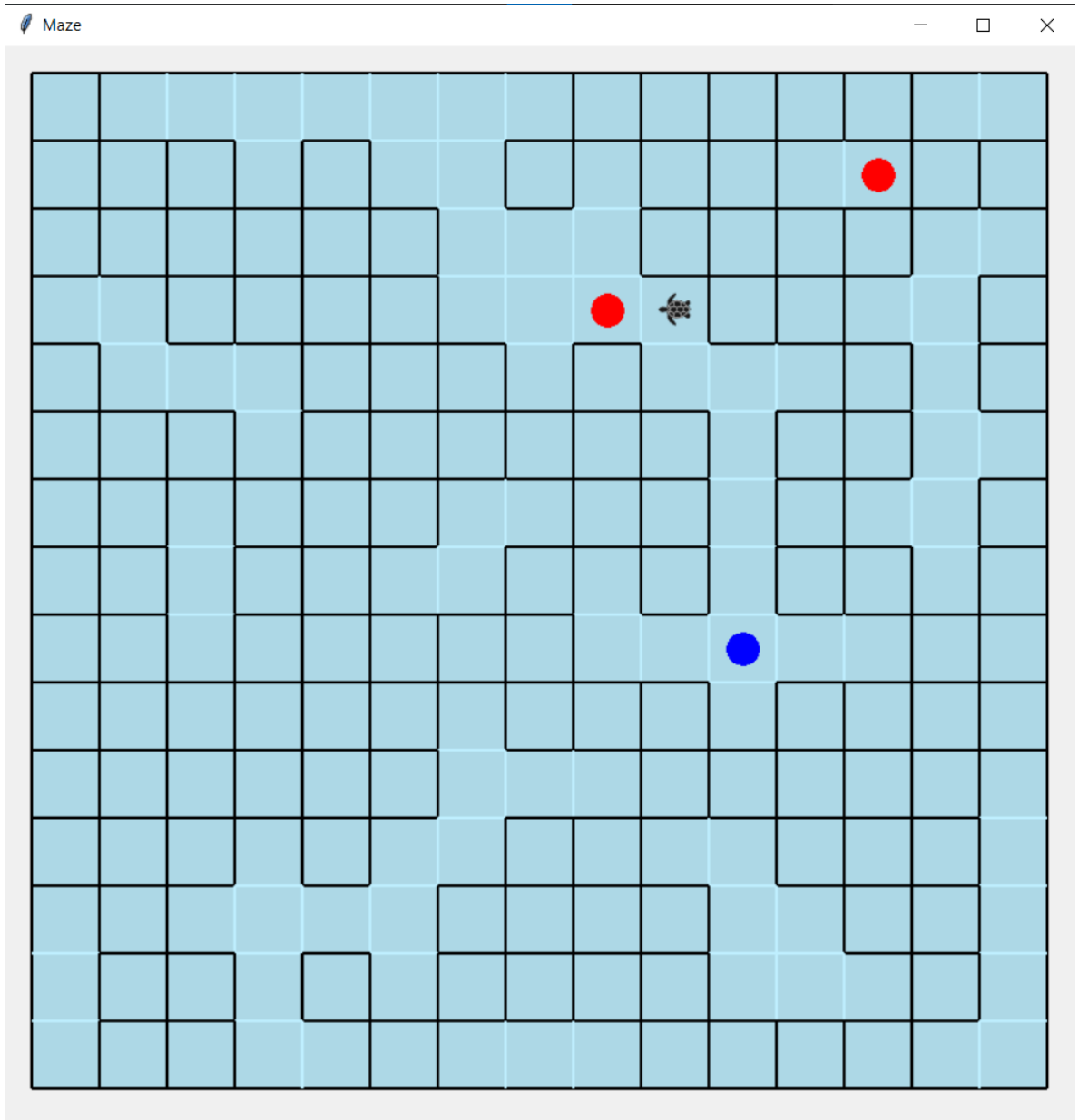

Figura 4. Prueba 1 con generación aleatoria de nodos, tortugas y obstáculos hasta un 70% de obstáculos.

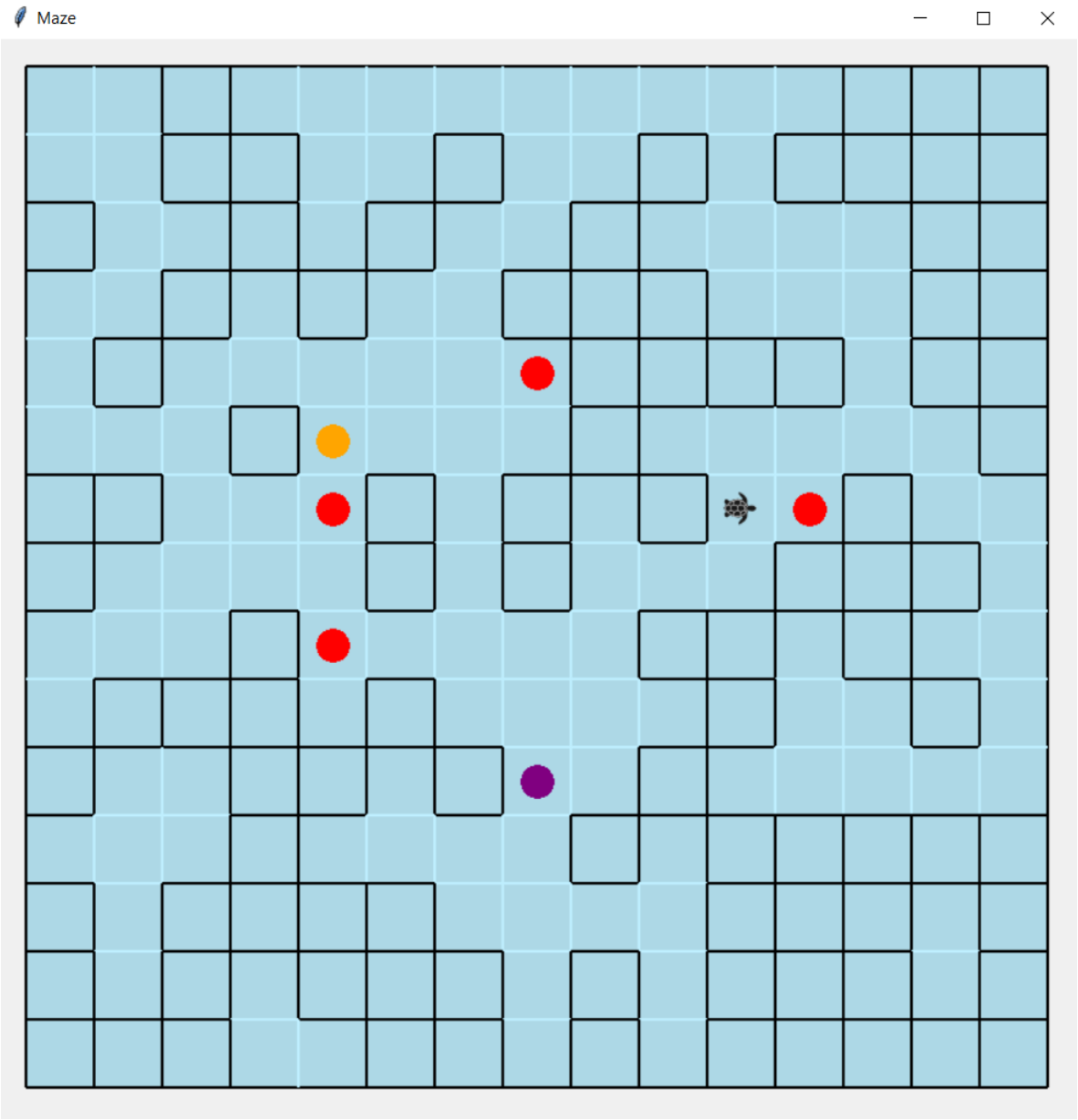

Figura 5. Prueba 2 con generación aleatoria de nodos, tortugas y obstáculos hasta un 70% de obstáculos.

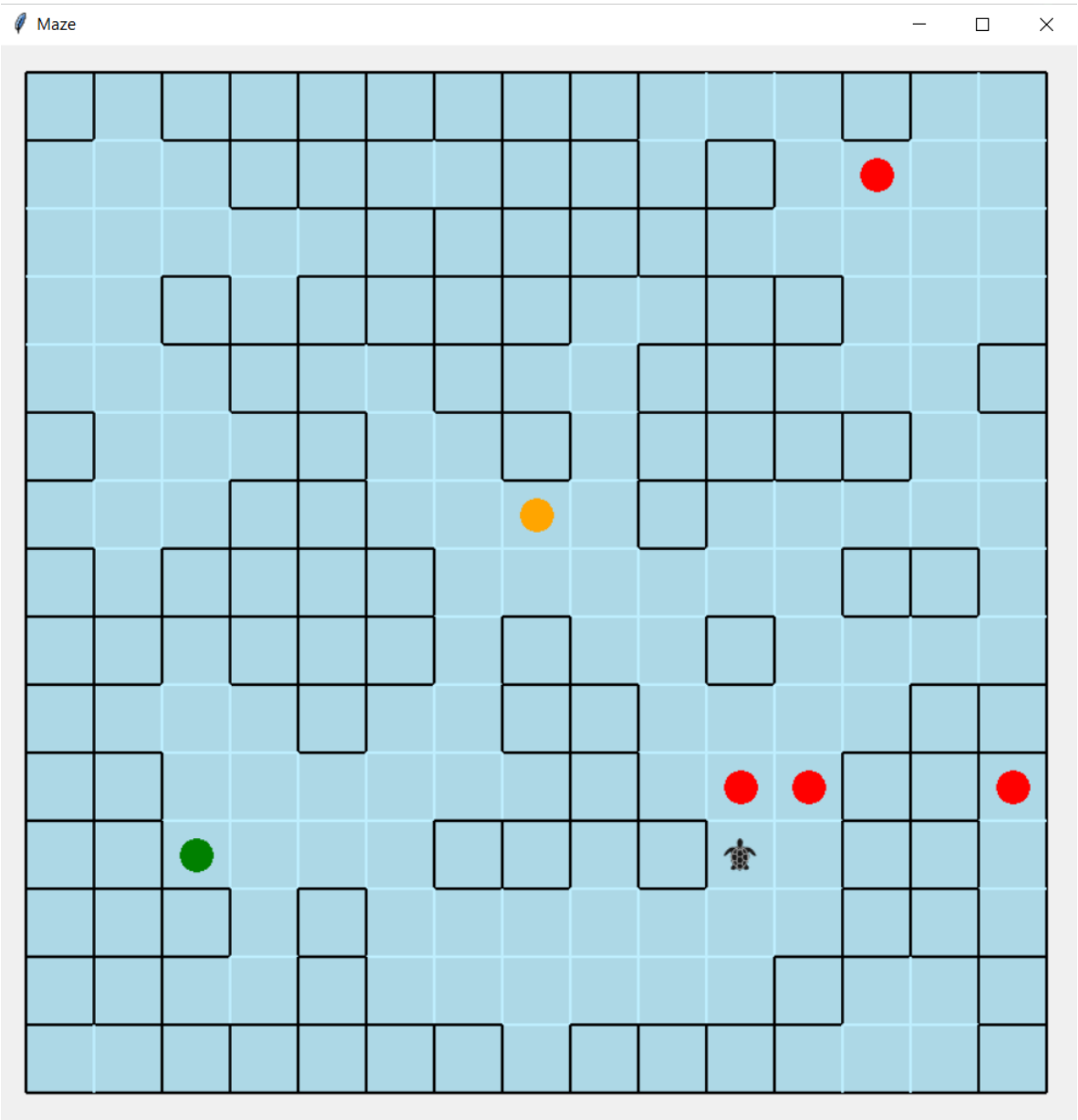

Figura 6. Prueba 3 con generación aleatoria de nodos, tortugas y obstáculos hasta un 70% de obstáculos.

Resultado: Se detectaron correctamente rutas inviables para tortugas afectadas por la desconexión.

Conclusiones

Rendimiento del Algoritmo: Dijkstra demostró ser una solución robusta y eficiente para grafos ponderados con pesos no negativos, incluso en escenarios con alta densidad de obstáculos.

Aplicaciones Prácticas

Este sistema tiene implementaciones a nivel profesional y de investigación en la vida real, tales como:

Planificación de rutas para vehículos autónomos.

"Path planning algorithms in the autonomous driving system: A comprehensive review"

https://doi.org/10.1016/j.robot.2024.104630

Cuantificación de la complejidad de la navegación en las redes de transporte

Optimización de rutas en redes de distribución (logística).

"Quantifying navigation complexity in transportation networks"

https://doi.org/10.1093/pnasnexus/pgad465

Una revisión de las tendencias recientes en la planificación del movimiento de robots industriales.

"A review of recent trend in motion planning of industrial robots"

https://doi.org/10.1007/s41315-023-00274-2

Código: se encuentra implementado en el archivo **robot_multiagente.py**